Phantom Crypter

Technical Documentation

Basic Execution Chain

Phantom wraps an encrypted .NET payload (or native EXE converted to Donut shellcode) into a single self-contained batch file. The batch uses VBS scripts via WScript.Shell to execute PowerShell with hidden window style 0. PowerShell decrypts (AES-256-CBC) and decompresses (GZip) the embedded payload stub, which then loads the final payload in-memory via Assembly.Load(byte[]). No files written to disk.

AMSI Bypass - Overview

Microsoft's AMSI (Anti-Malware Scan Interface) allows AV/EDR products to scan script content and .NET assemblies before execution. AmsiScanBuffer is the key export in amsi.dll that performs the scan. Bypassing it is essential for loading encrypted payloads.

AMSI Bypass - INT3 + VEH (Current)

This fork uses an INT3 (0xCC) breakpoint placed at the entry point of AmsiScanBuffer. When the CPU executes INT3, it raises EXCEPTION_BREAKPOINT (0x80000003). A Vectored Exception Handler (VEH) registered via AddVectoredExceptionHandler catches this exception and zeroes the return value at [sp+0x30] (x64) or [sp+0x18] (x86), causing AmsiScanBuffer to return AMSI_RESULT_CLEAN. Execution resumes at the original instruction.

Key advantages over the original VirtualProtect-based patch:

- No VirtualProtect call on amsi.dll (avoids Behavior:Win32/SuspAmsiPatch.F/K)
- No memory page permission changes (less suspicious)
- No hardcoded RVAs - uses GetProcAddress to locate AmsiScanBuffer
- Works across Windows 10 and 11 without version-specific offsets

AMSI Bypass - Dual Context

AMSI is called independently in two separate runtimes:
1. PowerShell script context - When PowerShell executes the decryption script, AmsiScanBuffer scans the script content. The INT3/VEH bypass in the PowerShell-level code (GetINT3Code in StubGen.cs) handles this.
2. C# stub context - When the decrypted assembly is loaded via Assembly.Load(), CLR/COM interop triggers a second AmsiScanBuffer call. The stub (Stub.cs) has its own independent INT3/VEH bypass for this.

ETW Bypass

Event Tracing for Windows (ETW) logs runtime behavior to Event Tracing sessions that EDR products consume. The stub patches ntdll!EtwEventWrite via VirtualProtect with a simple ret (0xC3) on x64 or ret 0x14 (0xC2 0x14 0x00) on x86, preventing any ETW logging from the process.

Hidden Execution

The batch creates two VBS scripts via WScript.Shell. The first VBS relaunches the batch hidden (window style 0). The batch detects this relaunch via a marker argument, creates a second VBS that runs 'powershell -File script.ps1' also hidden. This produces two brief console flashes but no persistent windows.

Original BStub Approach (c5hackr)

The original c5hackr/Phantom compiled a separate BStub.exe that patched AmsiScanBuffer via VirtualProtect + Marshal.Copy, overwriting the prologue with 'mov eax, 0x80070057; ret' (AMSI_RESULT_CLEAN). It used hardcoded RVAs for Win10 (0x180003860) and Win11 (0x180008260) with ASLR rebasing via GetModuleHandle. This approach was effective but triggered Behavior:Win32/SuspAmsiPatch.F/K due to the VirtualProtect call on amsi.dll.

Build Process

1. Read input .exe, determine file type (.NET x64/x86 or native x64/x86)
2. For native: convert to Donut shellcode (embedded donut.exe)
3. Encrypt payload with AES-256-CBC + GZip compression
4. Generate C# stub (Stub.cs) with INT3 VEH AMSI bypass, compile via CSharpCodeProvider
5. Encrypt the stub binary with AES-256-CBC + GZip (outer encryption)
6. Generate batch file with embedded encrypted payloads
7. Output: single .bat file, no dependencies

Defender Detection Status

Tested on Windows 11 24H2 with real-time protection ON. Deployed from a non-excluded directory with no Defender exclusions. Zero detections with all features enabled (hidden + selfdelete + runas + startup + antidebug + antivm).